This lab session covers Django Models (Part 2 – Q operator, sorting objects, aggregation functions, and multi-models). By the end of this session, students should have a very good understanding of how to build advanced queries with multi-model query in Django.

Pre-lab Preparation:

1. The table book as in lab7.
2. The fake data from mockaroo.com website or any other methods.

Lab Activities:

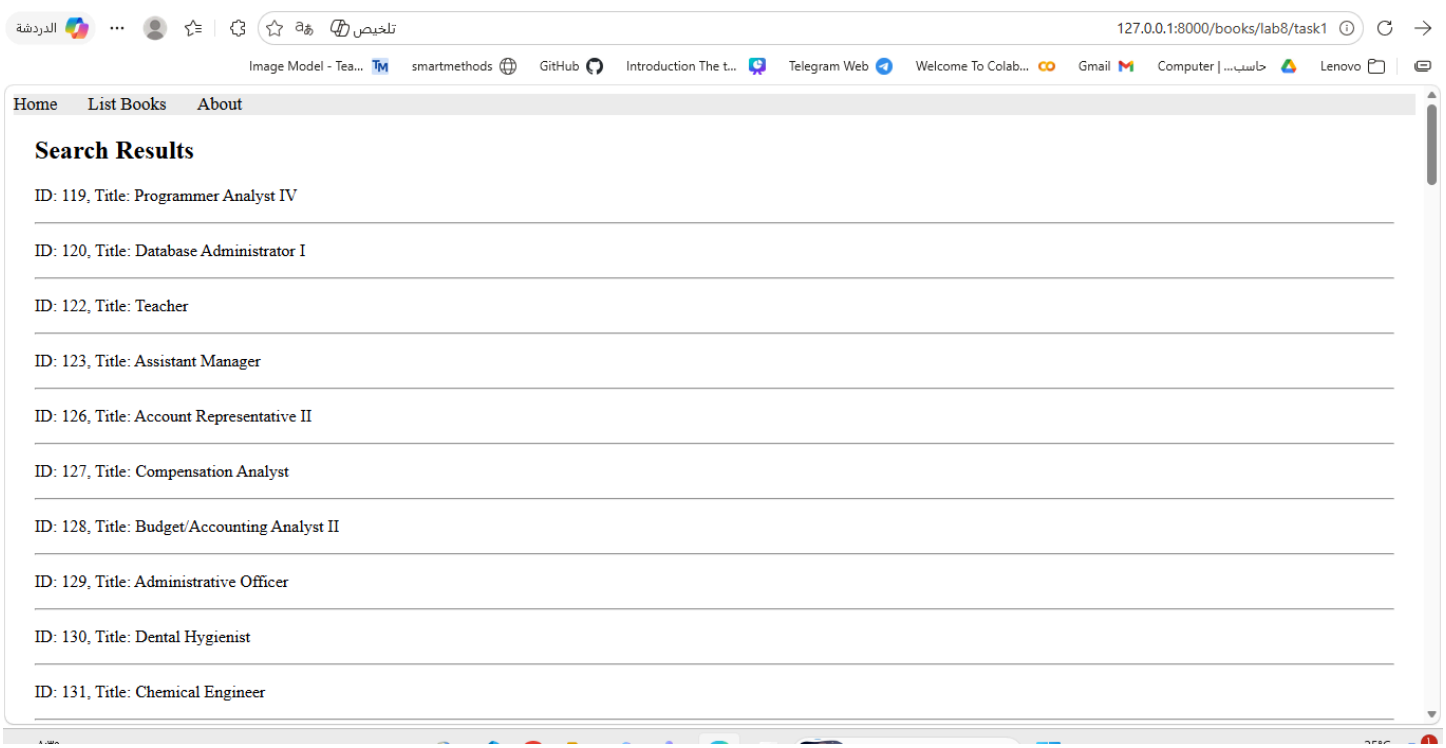
Task 1: Create a URL '/books/lab8/task1' with any necessary HTML file and view function to list books that have price less than or equal 80 using Q operator.

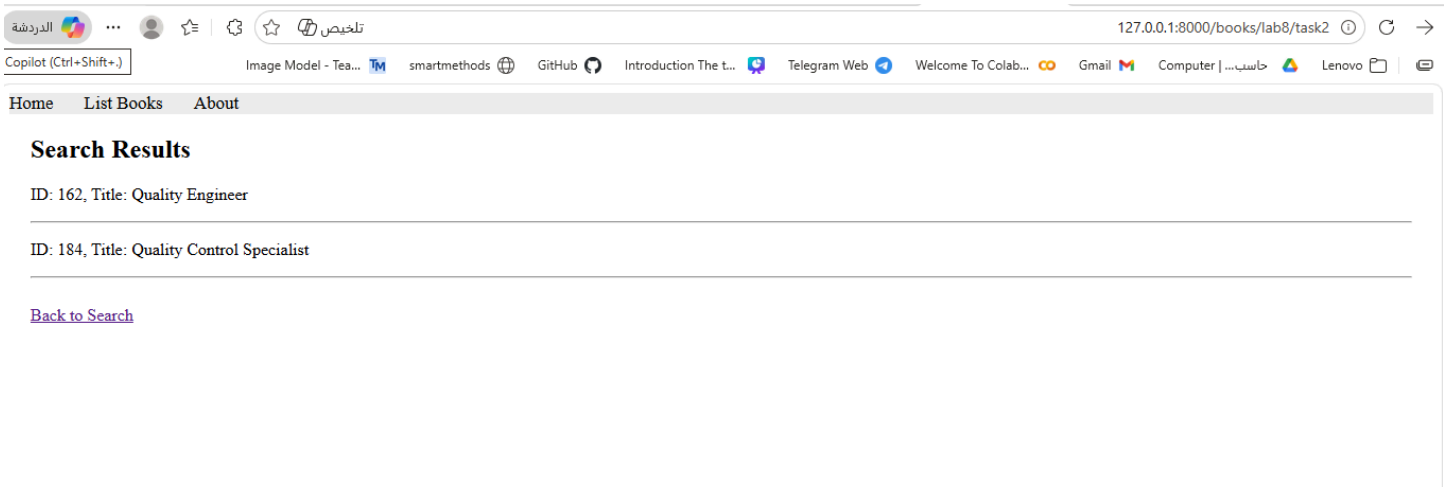
Task 2: Create a URL '/books/lab8/task2', HTML file, and view function to list books that have editions higher than three and either the title or author of the book contains the two adjacent letters 'qu' by combining two or more Q operator (through "&" or " | ").

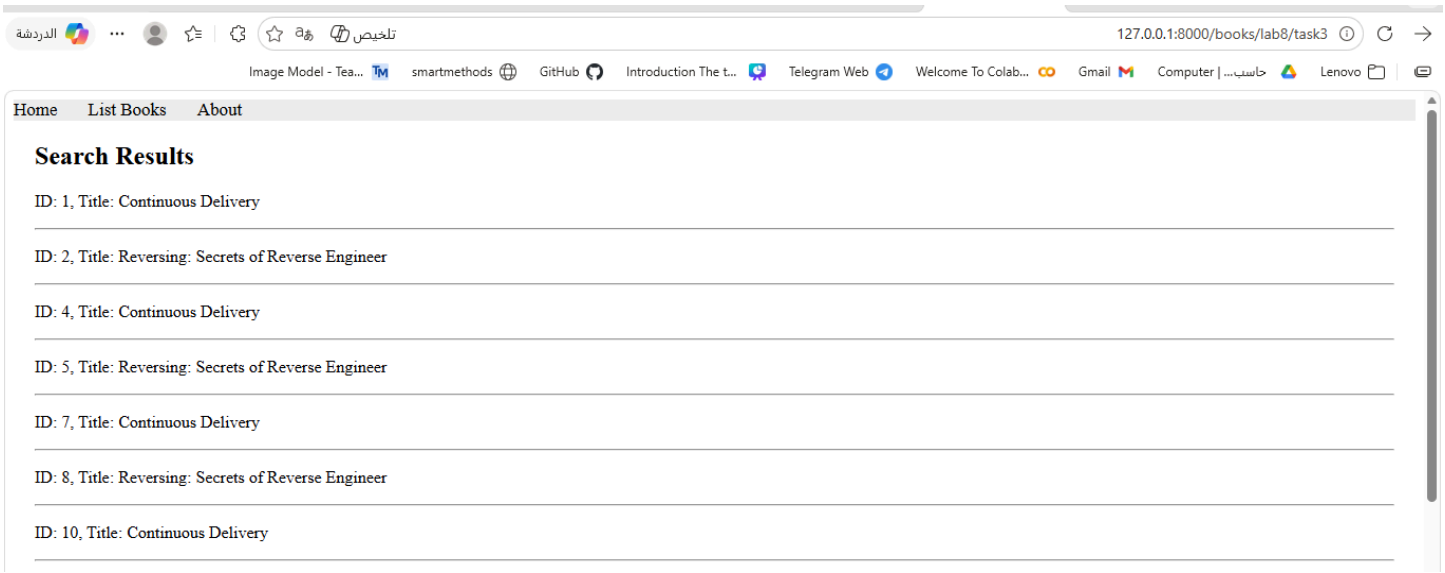
Task 3: (opposite to Task 2) Create a URL '/books/lab8/task3', HTML file, and view function to list books that have no editions higher than three and either the title or author of the book that does not contain the two adjacent letters "qu" by combining two or more Q operator (through "&" or " | ", beside ~ operator).

> Users > Lenovo > djangoenv > DjangoProjects > libraryproject > apps > bookmodule > views.py > task2

```python
from django.db.models import Q, Count, Sum, Avg, Max, Min
from django.shortcuts import render
from django.http import HttpResponse
from django.shortcuts import render
from .models import Book

from .models import Book

# Task 1:
def task1(request):
    books = Book.objects.filter(Q(price__lte=80))
    return render(request, 'bookmodule/bookList.html', {'books': books})

# Task 2:
def task2(request):
    books = Book.objects.filter(Q(edition__gt=3) & (Q(title__icontains='qu') | Q(author__icontains='qu')))
    return render(request, 'bookmodule/bookList.html', {'books': books})

# Task 3:
def task3(request):
    books = Book.objects.filter(~Q(edition__gt=3) & ~Q(title__icontains='qu') & ~Q(author__icontains='qu'))
    return render(request, 'bookmodule/bookList.html', {'books': books})
```



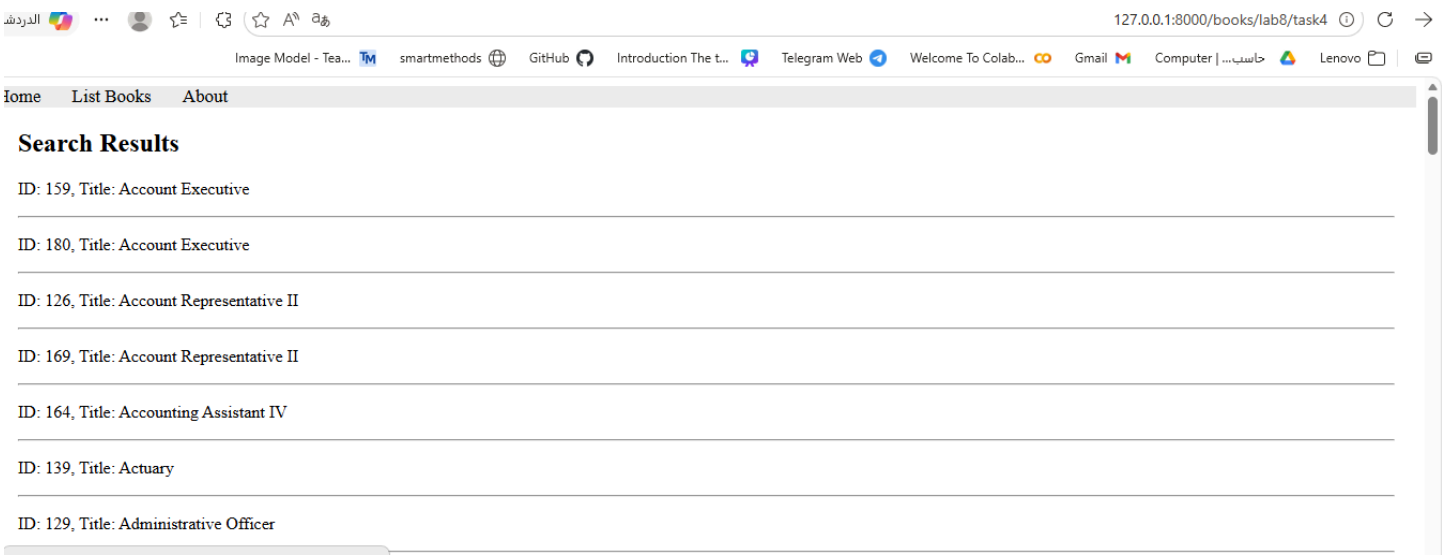


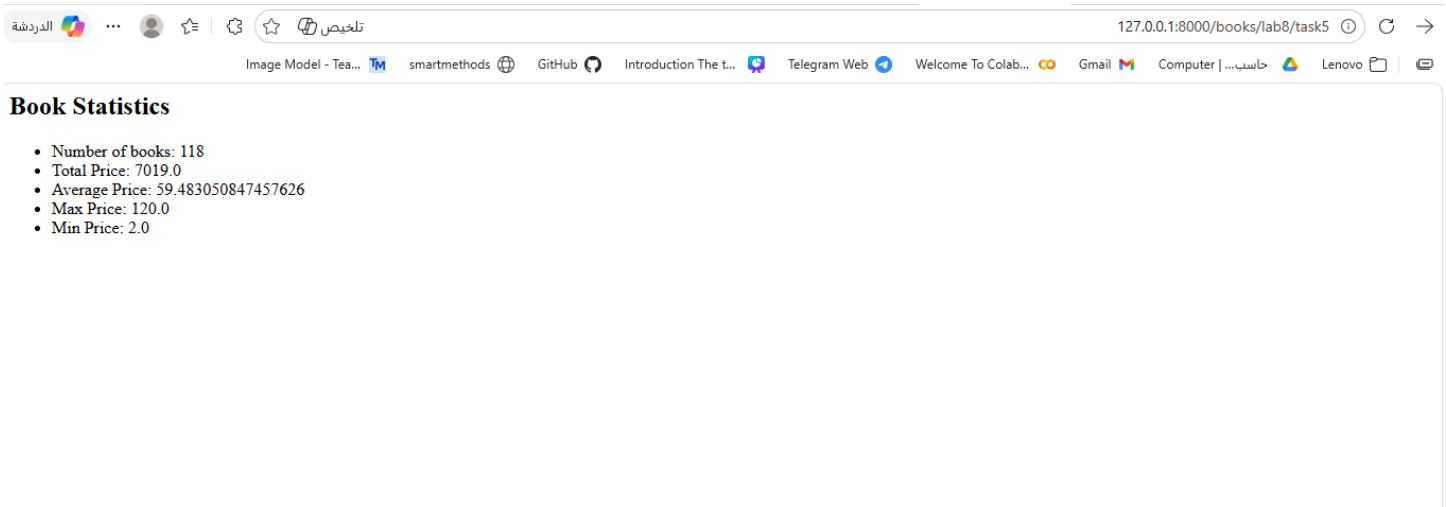


Task 4: Create a URL '/books/lab8/task4', HTML file, and view function to list books and order by their titles using the function "order_by".

Task 5: Create a URL '/books/lab8/task5', HTML file, and view function to display the number of books, total price of all books, average price, maximum price, and minimum price using aggregation functions.

```python
# Task 4:
def task4(request):
    books = Book.objects.order_by('title')
    return render(request, 'bookmodule/bookList.html', {'books': books})

# Task 5:
def task5(request):
    stats = Book.objects.aggregate(
        total_books=Count('id'),
        total_price=Sum('price'),
        avg_price=Avg('price'),
        max_price=Max('price'),
        min_price=Min('price')
    )
    return render(request, 'bookmodule/task5.html', {'stats': stats})
```





Task 6: Create a database scheme (through models in Django)

Note that instead of creating another app to handle student applications, you may exploit existing apps, like user or book modules

views.py 4 ● models.py 4 ×

C: > Users > Lenovo > djangoenv > DjangoProjects > libraryproject > apps > bookmodule > models.py >

```python
from django.db import models

# Create your models here.

from django.db import models

class Address(models.Model):
    city = models.CharField(max_length=100)
    def __str__(self):
        return self.city

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    address = models.ForeignKey(Address, on_delete=models.CASCADE)

```

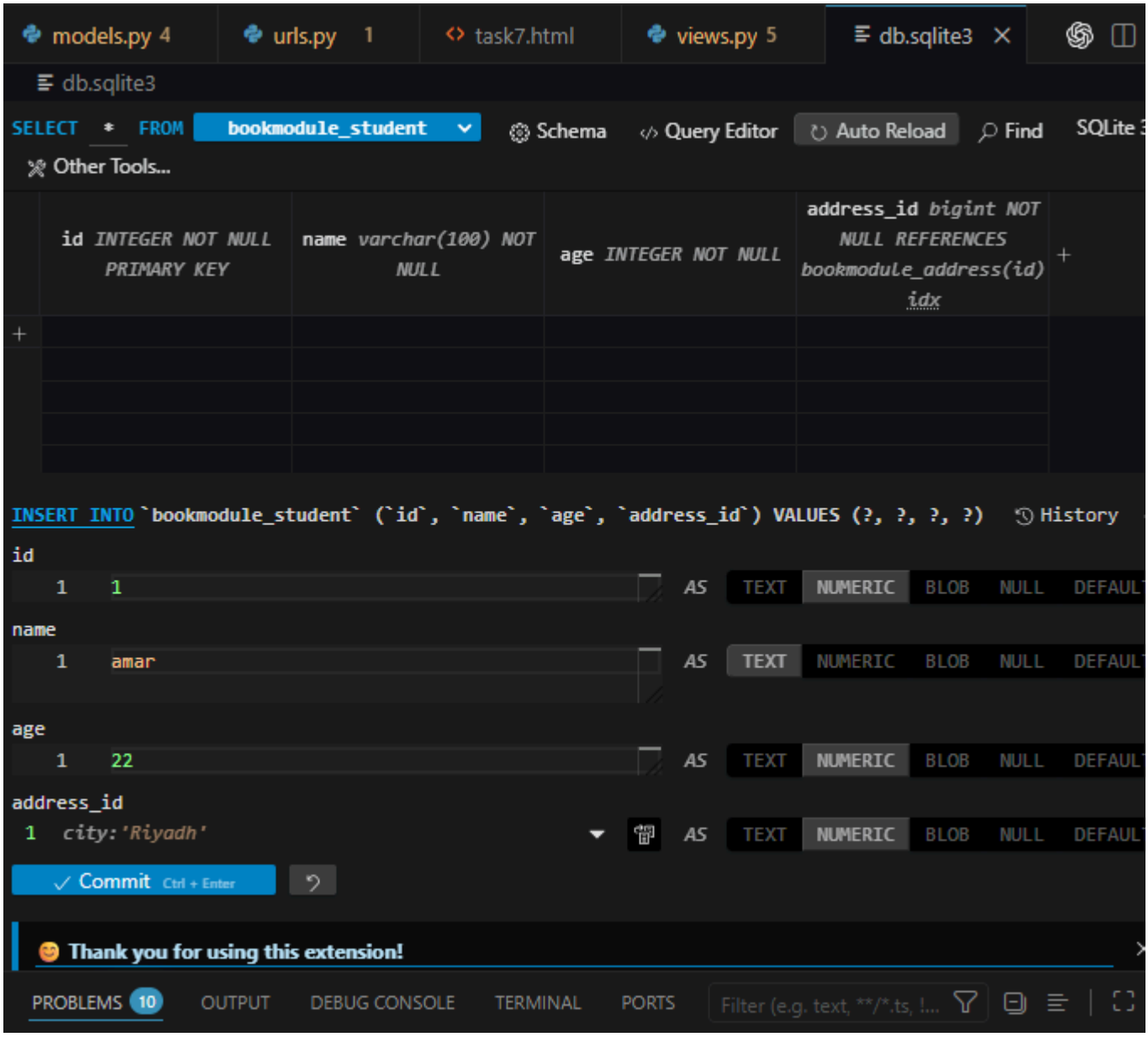
Student

ID	as an integer (this is done automatically by Django)

name	as a string
age	as an integer
address	as a foreign key to table address

Address

ID	as an integer (this is done automatically by Django)
city	as a string


models.py 4
urls.py 1
task7.html
views.py 5
db.sqlite3
db.sqlite3
SELECT * FROM bookmodule_student
Schema
Query Editor
Auto Reload
Find
SQLite 3
Other Tools...
id INTEGER NOT NULL PRIMARY KEY
name varchar(100) NOT NULL
age INTEGER NOT NULL
address_id bigint NOT NULL REFERENCES bookmodule_address(id) idx
INSERT INTO `bookmodule_student` (`id`, `name`, `age`, `address_id`) VALUES (?, ?, ?, ?)
History
id
1 1
name
1 amar
age
1 22
address_id
1 city:'Riyadh'
AS TEXT NUMERIC BLOB NULL DEFAULT
Commit Ctrl + Enter
Thank you for using this extension!
PROBLEMS 10
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Filter (e.g. text, **/*.ts, !...

SELECT * FROM bookmodule_student | Schema | Query Editor | Auto Reload | Find | SQLite 3.5

Other Tools...

	id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT	name varchar(100) NOT NULL	age INTEGER NOT NULL	address_id bigint NOT NULL REFERENCES bookmodule_address(id) idx
1	1	amar	22	1 city:'Riyadh'
2	2	salah	40	2 city:'Qassim '

INSERT INTO `bookmodule_student` (`name`, `age`, `address_id`) VALUES (?, ?, ?) History

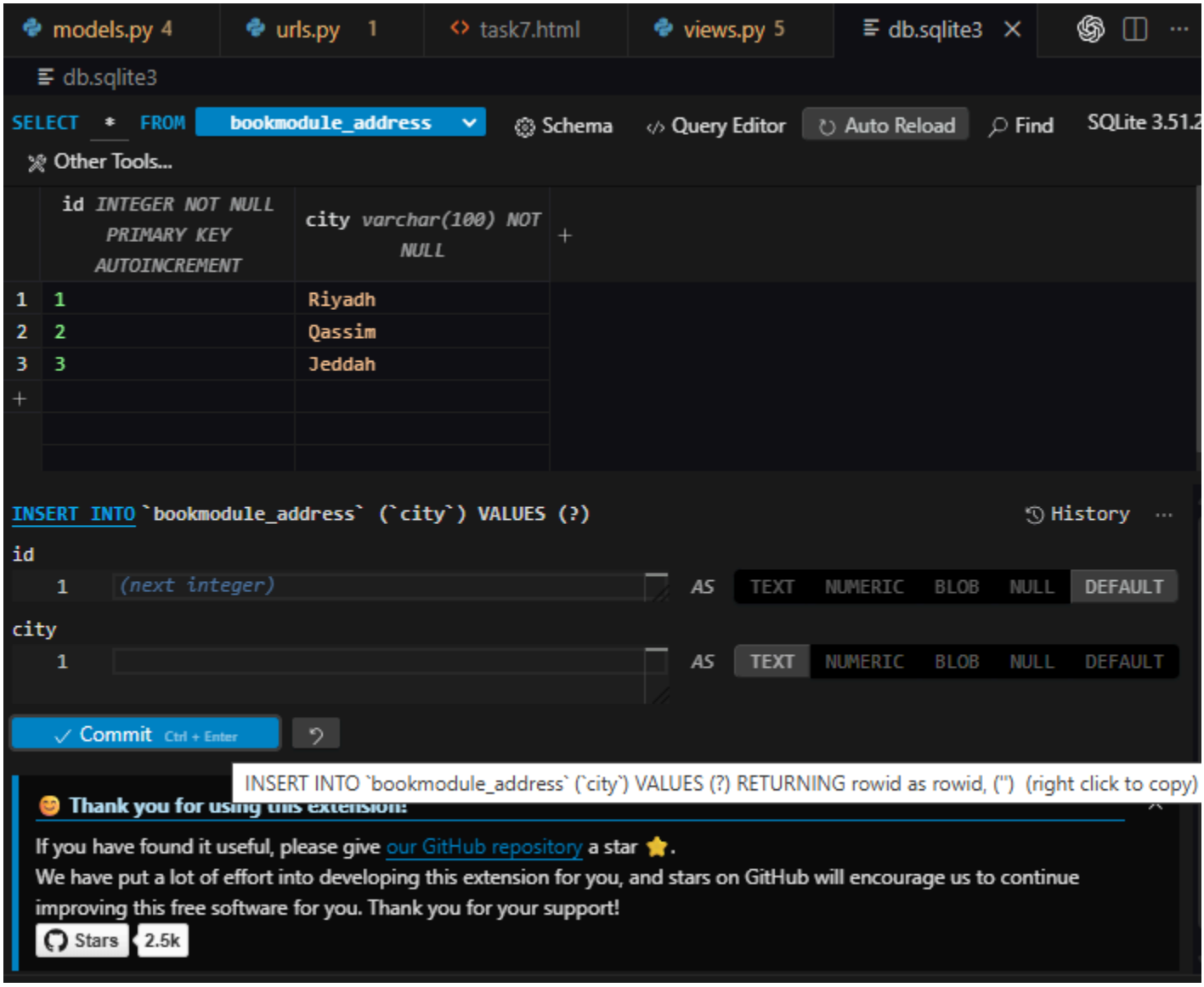
id

1 (next integer) AS TEXT NUMERIC BLOB NULL DEFAULT

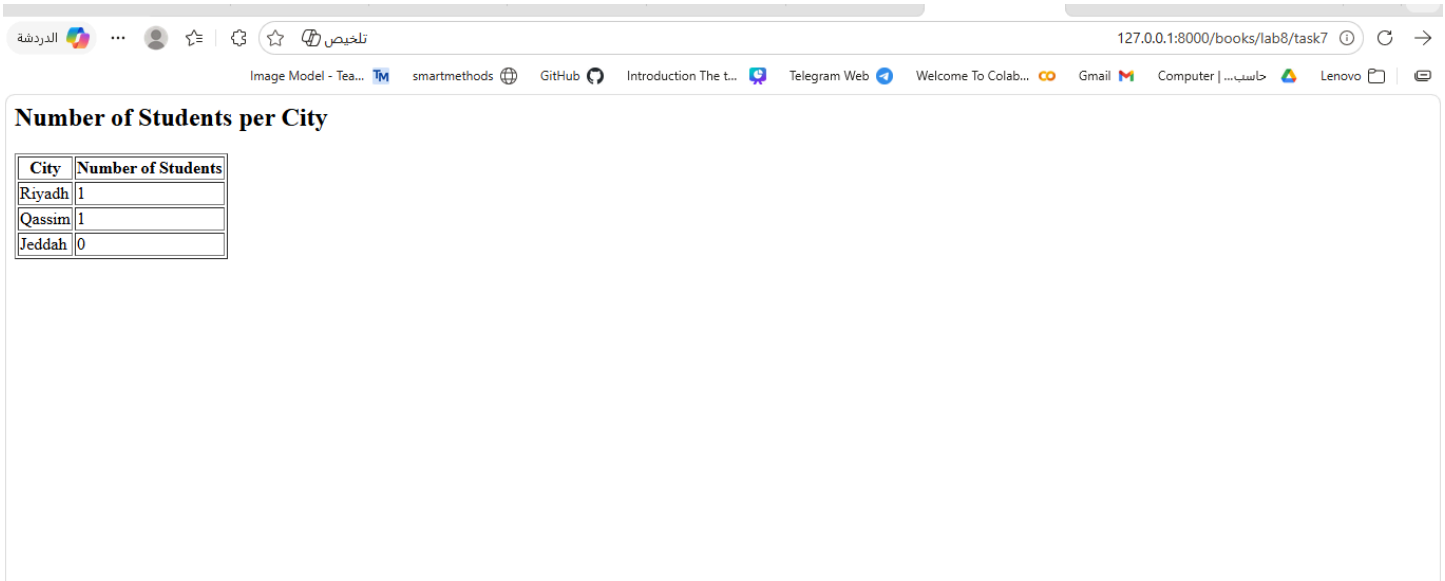
name

1 AS TEXT NUMERIC BLOB NULL DEFAULT

age



Task7: Create a page that shows the number of students in each city.


127.0.0.1:8000/books/lab8/task7
Image Model - Tea...
smartmethods
GitHub
Introduction The t...
Telegram Web
Welcome To Colab...
Gmail
Computer | ...حاسب
Lenovo
Number of Students per City
City
Number of Students
Riyadh
1
Qassim
1
Jeddah
0